



Malaysian Institute of Information Technology

IIB43203 - Cloud Computing

Group Project Report

AWS Cloud Storage System

Prepared by

No.	Name	Student ID(s)
1.	Nur Syakirah Binti Md Asrul Effenddy	52227124838
2.	Nur Hanis Farzana Binti Mohd Zamani	52227124897
3.	Putra Danish Nur Qayyum Bin Nordin	52227124342

Prepared for

Megat Norulazmi Megat Mohamed Noor

Table of Contents

1. INTRODUCTION.....	3
1.1 Project Background.....	3
1.2 Project Objectives	3
1.3 Project Scope	4
2. TEAM MEMBERS & TASK DISTRIBUTION	5
3. SYSTEM ARCHITECTURE.....	6
3.1 System Architecture Diagram	6
3.2 Architecture Components	7
3.3 System Workflow	8
4. AWS SERVICES.....	9
5. WEB APPLICATION FEATURES	11
6. IMPLEMENTATION.....	12
6.1 Amazon Resources Configuration	12
6.2 Amazon EC2 Setup	16
6.3 Connect to Amazon EC2 Instance.....	18
6.4 Backend Application Development and Configuration.....	19
6.5 Backend Integration and Application Testing	21
6 FINAL SYSTEM TESTING	25
8 RESULTS	35
7 CONCLUSION.....	36
REFERENCES.....	37

1. INTRODUCTION

1.1 Project Background

Cloud computing enables applications and services to be hosted online and accessed through the internet from anywhere in the world. In this project, Amazon Web Services (AWS) was used to develop and deploy an AWS Cloud Storage System. The system allows users to upload, view, access, and delete files through a web-based interface, providing a simple and efficient cloud storage solution.

The application was developed using Node.js and Express.js and deployed on an Amazon EC2 instance. Uploaded files are stored securely in Amazon S3, while file metadata such as file name, file URL, upload date, and file identifier are stored in Amazon DynamoDB. AWS Identity and Access Management (IAM) was configured to manage permissions securely, and Security Groups were used to control network access to the cloud resources.

This project demonstrates the practical implementation of cloud computing concepts by utilizing AWS services such as Compute (Amazon EC2), Storage (Amazon S3), Database (Amazon DynamoDB), Networking (Security Groups and Public IP), and Security (IAM). The integration of these services provides a functional cloud-based file storage system that allows users to manage files efficiently through a web application.

1.2 Project Objectives

The objectives of this project are:

- To develop and deploy a web-based Cloud Storage System using AWS cloud services.
- To utilize Amazon EC2 as the compute service for hosting the web application.
- To implement Amazon S3 and Amazon DynamoDB for cloud storage and file metadata management.
- To configure AWS networking services, including VPC connectivity and Security Groups, to enable secure communication between users and cloud resources.
- To implement AWS Identity and Access Management (IAM) roles and permissions to provide secure access and management of AWS resources.

1.3 Project Scope

The scope of this project is to develop and deploy a web-based Cloud Storage System using Amazon Web Services (AWS). The system is designed to provide users with a simple platform for managing files through functionalities such as uploading, viewing, opening, and deleting files. Uploaded files are stored securely in cloud storage, while file metadata including file name, file URL, upload date, and file identifier are stored in a database for efficient management and retrieval.

The web application is hosted on an Amazon EC2 Amazon Linux server running Apache Web Server and PHP, while MySQL is used as the database management system to store and manage student records. Amazon S3 is utilized for cloud storage of project-related files and backup materials. AWS Security Groups are configured to control network access to the server, and AWS Identity and Access Management (IAM) is used to manage user permissions and secure access to AWS resources.

The project also covers cloud infrastructure setup, local development and testing, backend development, AWS service integration, file upload and management implementation, server configuration, AWS deployment, system testing, and documentation. The scope is limited to basic file management operations such as uploading, viewing, accessing, and deleting files. Advanced features such as user authentication, role-based access control, file sharing, file versioning, notifications, and mobile application integration are not included in this project.

2. TEAM MEMBERS & TASK DISTRIBUTION

Name	Role	Responsibility
Nur Hanis Farzana	Project Management and System Documentation Lead	Coordinated project planning and task management, prepared the project report, documented AWS implementation processes and services used, organized screenshots and supporting evidence, and prepared the project presentation and demonstration video.
Putra Danish	Web Application Development and AWS Deployment Lead	Developed the AWS Cloud Storage System using Node.js and Express.js, implemented file upload, viewing, and deletion functionalities, configured and managed the Amazon EC2 server, networking and IAM, integrated Amazon S3 and DynamoDB services, deployed the web application on AWS, and performed system testing and troubleshooting. Also, GitHub Setup.
Nur Syakirah	AWS Architecture and Cloud Infrastructure Lead	Designed the cloud system architecture, configured AWS services including IAM, Security Groups, Amazon S3, and DynamoDB, managed cloud infrastructure setup, assisted with deployment and server configuration, and validated the functionality and security of AWS resources and services.

3. SYSTEM ARCHITECTURE

3.1 System Architecture Diagram

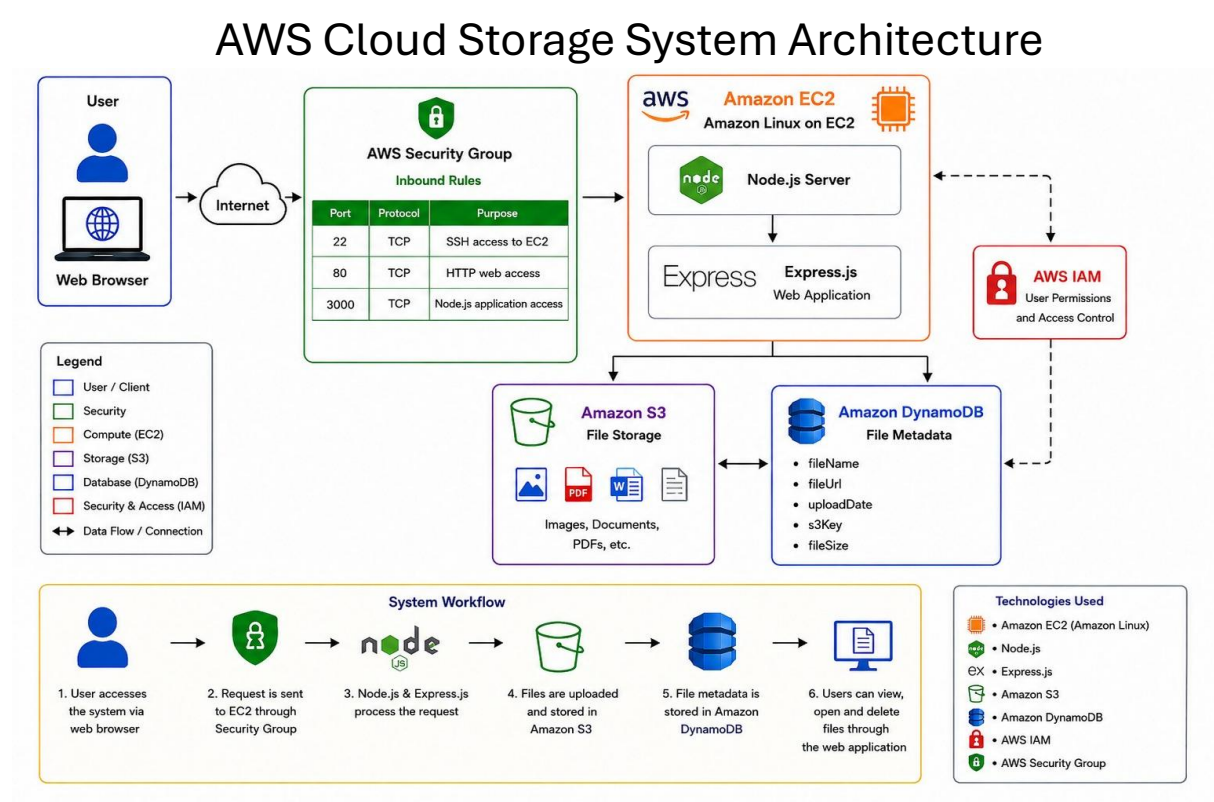


Figure 1: System Architecture of AWS Cloud Storage System

The AWS Cloud Storage System architecture is illustrated in Figure 1. Users access the system through a web browser using the public IPv4 address assigned to the Amazon EC2 instance. Requests are transmitted through the internet and routed to the AWS cloud environment, where the EC2 server processes the incoming requests.

The web application is hosted on an Amazon EC2 (Amazon Linux) server running a Node.js and Express.js application. The Node.js server is responsible for handling incoming HTTP requests and processing user interactions. Through the web interface, users can upload files, view uploaded files, access stored files, and delete files from the system.

When a user uploads a file, the Node.js application processes the upload request and stores the file in Amazon S3. Amazon S3 serves as the cloud storage service, providing secure and scalable storage for files such as images, documents, PDFs, and other supported file types. Each uploaded file is assigned a unique identifier to ensure efficient storage and retrieval.

In addition to storing files in Amazon S3, the application stores file metadata in Amazon DynamoDB. The metadata includes information such as file name, file URL, upload date, file identifier, and S3 storage key. Whenever a user views or manages files, the application retrieves the corresponding metadata from DynamoDB and displays it through the web interface.

AWS Identity and Access Management (IAM) is used to manage permissions and access control for AWS resources. IAM ensures that only authorized services and users can access Amazon S3, DynamoDB, and the EC2 instance. Furthermore, AWS Security Groups act as a virtual firewall by controlling inbound and outbound network traffic to the EC2 server, allowing secure access through protocols such as SSH and HTTP.

Overall, the system architecture demonstrates the integration of AWS Compute (Amazon EC2), Storage (Amazon S3), Database (Amazon DynamoDB), Networking (Public IP Address and Security Groups), and Security (IAM) services to deliver a functional cloud-based file storage system that can be accessed and managed through the internet.

3.2 Architecture Components

- User Web Browser
- Internet
- AWS Security Group
- Amazon EC2 (Amazon Linux)
- Node.js Server
- Express.js Web Application
- Amazon S3 Storage
- Amazon DynamoDB Database
- AWS IAM

3.3 System Workflow

1. User accesses the Cloud Storage System through a web browser using the public IP address of the Amazon EC2 instance.
2. The request is transmitted through the internet and allowed by the AWS Security Group.
3. The Node.js server running on the Amazon EC2 instance receives and processes the request.
4. For file uploads, the application stores files in Amazon S3 cloud storage.
5. File metadata such as file name, file URL, upload date, and file identifier are stored in Amazon DynamoDB.
6. When users view or manage files, the application retrieves metadata from DynamoDB and displays the information through the web interface.
7. Users can open or delete files, and the corresponding updates are performed in Amazon S3 and DynamoDB.
8. AWS IAM and Security Groups manage access permissions and ensure secure communication between AWS resources and users.

4. AWS SERVICES

This project utilizes several Amazon Web Services (AWS) components to develop, deploy, secure, and manage the AWS Cloud Storage System web application. Amazon EC2 serves as the primary cloud server for hosting the Node.js and Express.js web application, while additional AWS services such as Amazon S3, Amazon DynamoDB, AWS Security Groups, and AWS Identity and Access Management (IAM) provide storage, database, networking, and security functionalities. Together, these services demonstrate the practical implementation of cloud computing concepts including compute, storage, database, networking, and security.

AWS Requirement	Service/Component Used	Description
Compute	Amazon EC2	Used to create a virtual Amazon Linux to host the Student Management System web application.
Storage	Amazon S3	Used to store uploaded files such as images, documents, PDFs, and other supported file types in cloud storage.
Networking	Public IP Address and Security Group	Used to provide secure access to the web application through the internet and control inbound and outbound network traffic.
Database	Amazon DynamoDB	Used to store and manage file metadata including file name, file URL, upload date, file identifier, and storage key information.
Security	AWS IAM	Used to manage user permissions and securely control access to AWS resources and services.

Amazon EC2 serves as the main cloud server for the project. The EC2 instance runs Amazon Linux Server and hosts the Node.js and Express.js web application, which together provide the environment required to operate the AWS Cloud Storage System. The web application files are deployed on the EC2 instance and can be accessed through the EC2 public IPv4 address.

Networking is configured using the EC2 public IP address and AWS Security Groups. Security Groups function as a virtual firewall by controlling inbound and outbound traffic to the server. Rules were configured to allow SSH access for administration purposes and web application access through the configured application port.

Amazon DynamoDB is used as the database management service for the project. It stores file metadata generated by the application, including file names, file URLs, upload dates, file identifiers, and storage keys. The Node.js application communicates directly with DynamoDB to store, retrieve, and manage file information.

Amazon S3 is used as the cloud storage service to store uploaded files such as images, documents, PDFs, and other supported file types. The S3 bucket provides secure, scalable, and highly available storage within the AWS cloud environment.

AWS Identity and Access Management (IAM) is used to manage user permissions and secure access to AWS resources. IAM ensures that only authorized users and services can access and manage AWS resources such as EC2, S3, and DynamoDB, helping to maintain the security of the cloud infrastructure.

Overall, the project demonstrates the integration of AWS Compute (Amazon EC2), Storage (Amazon S3), Database (Amazon DynamoDB), Networking (Public IP Address and Security Groups), and Security (IAM) services to deliver a fully functional cloud-based file storage system that can be accessed and managed through the internet.

5. WEB APPLICATION FEATURES

The AWS Cloud Storage System provides a web-based interface that enables users to manage files efficiently through cloud storage services. The system is designed to provide simple and user-friendly functionalities for uploading, viewing, accessing, and deleting files stored in the AWS cloud environment.

No.	Feature	Description
1.	Home Page	Displays the main interface of the Cloud Storage System and provides navigation to available functions.
2.	Upload File	Allows users to upload files such as images, documents, and PDFs to Amazon S3 cloud storage.
3.	Upload Success Page	Displays confirmation and file details after a successful upload operation.
4.	View Files	Displays a list of uploaded files retrieved from Amazon DynamoDB metadata records.
5.	Open File	Allows users to access and view files stored in Amazon S3 through generated file URLs.
6.	Delete File	Allows users to remove files from Amazon S3 and delete the corresponding metadata from Amazon DynamoDB.

6. IMPLEMENTATION

The AWS Cloud Storage System is a web-based application that allows users to store and manage files using cloud computing services. The system enables users to upload, view, access, and delete files through a simple and user-friendly interface. By integrating Amazon EC2, Amazon S3, Amazon DynamoDB, IAM, and Security Groups, the application provides a secure and efficient cloud-based file management solution that can be accessed through the internet.

6.1 Amazon Resources Configuration

a. Amazon S3 Bucket Creation:

Amazon S3 was configured to provide cloud storage for the AWS Cloud Storage System. A new S3 bucket was created with the required settings, including the AWS region and bucket name. The bucket is used to store files uploaded by users through the web application.

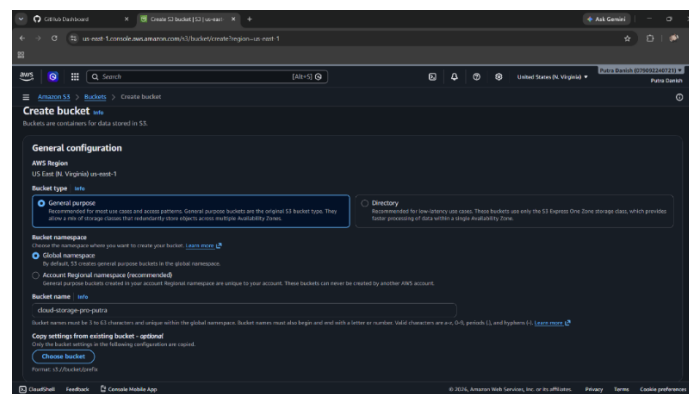


Figure 2: Amazon S3 Bucket Creation

Amazon S3 bucket was created to provide cloud storage for the AWS Cloud Storage System. The bucket was configured with the required settings, including the AWS region and bucket name, to store files uploaded through the web application.

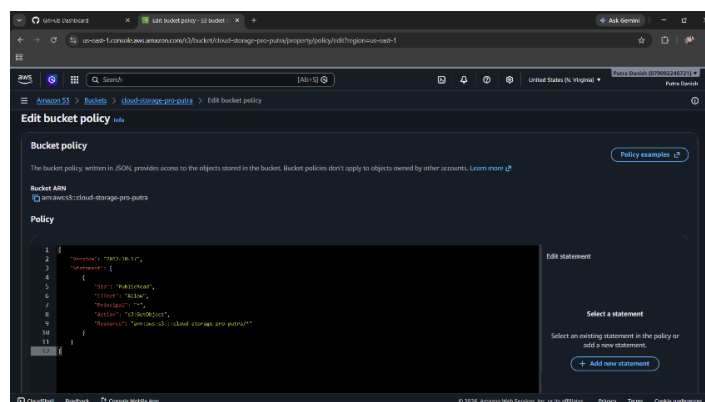


Figure 3: Amazon S3 Bucket Policy Configuration

The S3 bucket policy was configured to control access to stored files.

b. DynamoDB Table Creation:

Amazon DynamoDB is used to store file metadata instead of the actual files. The actual files are stored in Amazon S3, while DynamoDB keeps information about each uploaded file. When a user uploads a file, the file is stored in Amazon S3 and its metadata is saved in DynamoDB. This allows the system to efficiently view, retrieve, and delete files through the web application.

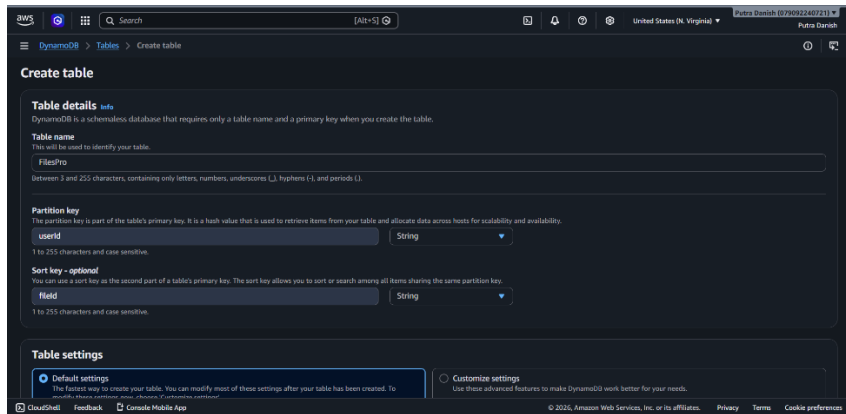


Figure 4: Amazon DynamoDB Setup

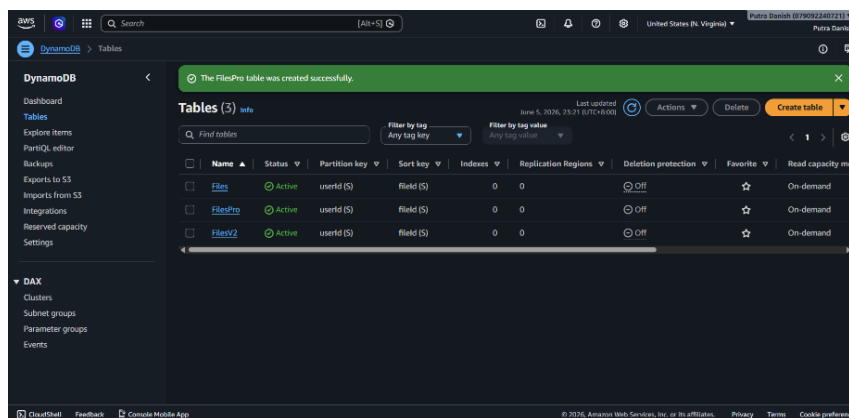


Figure 5: Amazon DynamoDB Table Creation

Configuration:

- Table Name: FilesPro
- Partition Key: userId (String)
- Sort Key: fileId (String)

Stored Information:

- User ID
- File ID
- File Name
- File URL
- S3 Key
- Upload Date

c. IAM Role Creation:

AWS Identity and Access Management (IAM) Role is used to grant the EC2 instance permission to access AWS services securely without storing AWS Access Keys in the application.

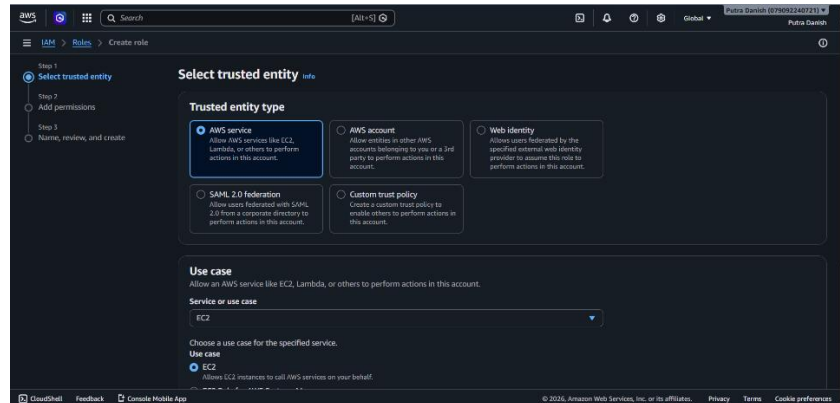


Figure 6: AWS IAM Configuration

AWS IAM was configured to control access to AWS EC2 services.

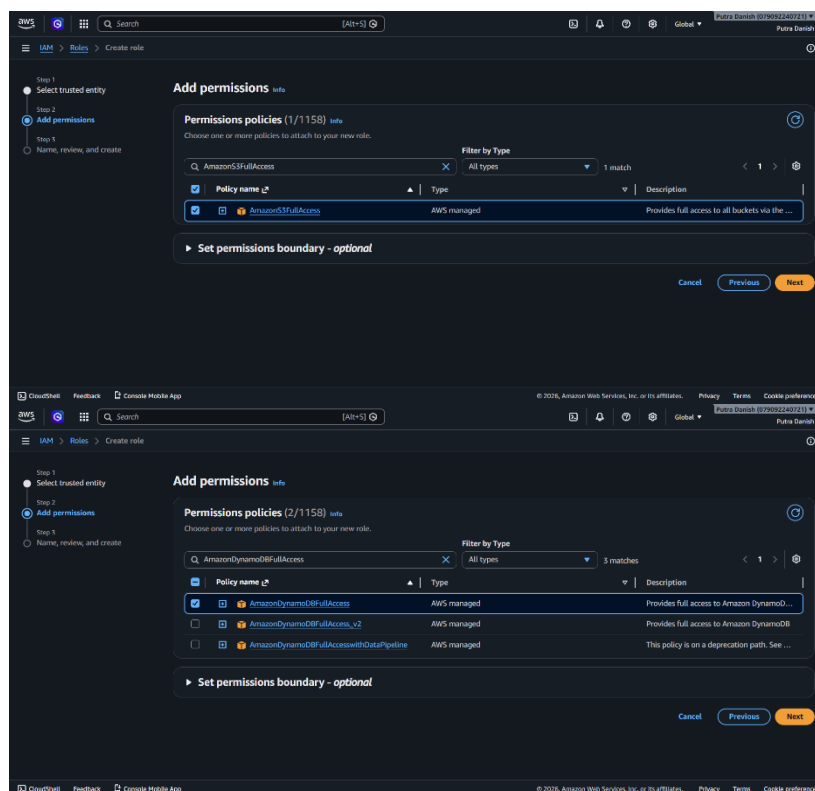


Figure 7: IAM Role Permission Configuration

IAM role permissions were configured to allow access to Amazon DynamoDB.

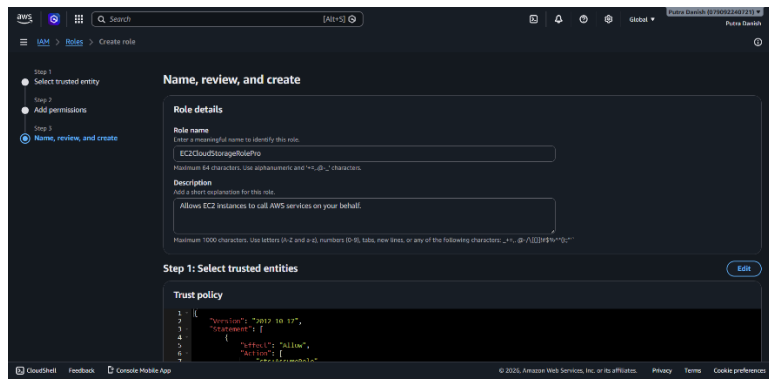


Figure 8: IAM Role Name

An IAM role name was created as EC2CloudStorageRolePro.

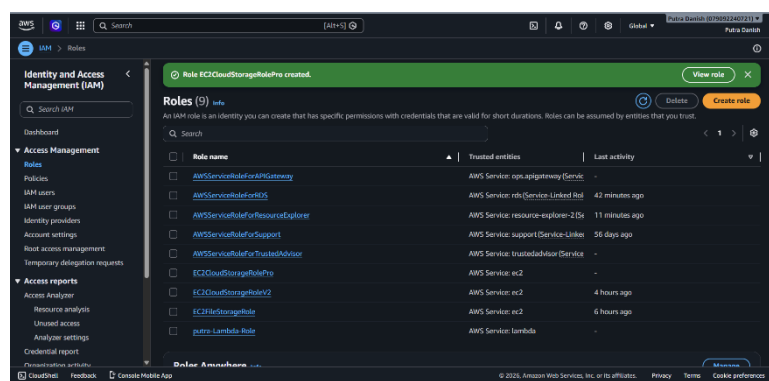


Figure 9: IAM Role Creation Complete

The IAM role allows the Node.js application running on EC2 to upload and delete files in Amazon S3 and store, retrieve, and delete metadata in DynamoDB. This improves security by using temporary AWS credentials automatically provided to the EC2 instance.

Configuration:

- Role Name: EC2CloudStorageRolePro
- Trusted Entity: EC2
- Attached Policies: AmazonS3FullAccess and AmazonDynamoDBFullAccess

6.2 Amazon EC2 Setup

Amazon EC2 was accessed through the AWS Management Console to create and manage the virtual server used for hosting the AWS Cloud Storage System.

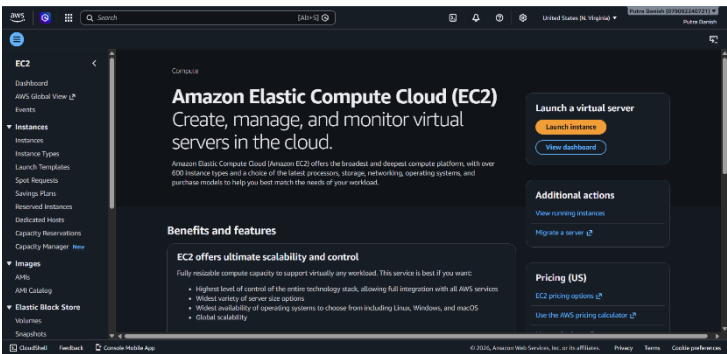


Figure 10: Amazon EC2 Instance Configuration

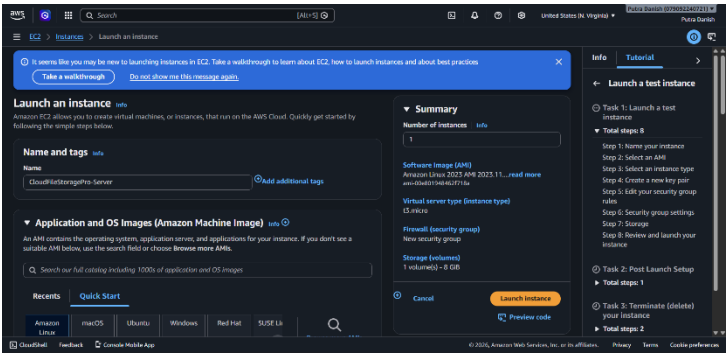


Figure 11: Amazon EC2 Instance Name

An Amazon EC2 instance was configured and prepared for deployment.

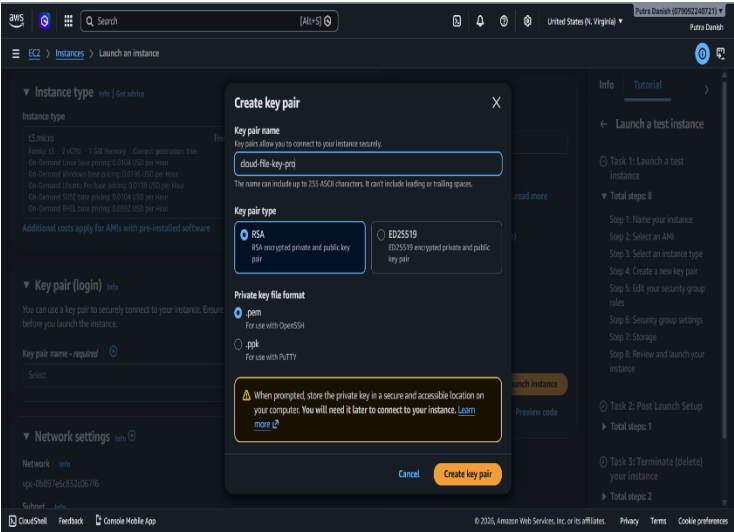


Figure 12: Amazon EC2 Key Pair Configuration

A key pair was created and configured to enable secure access to the EC2 instance.

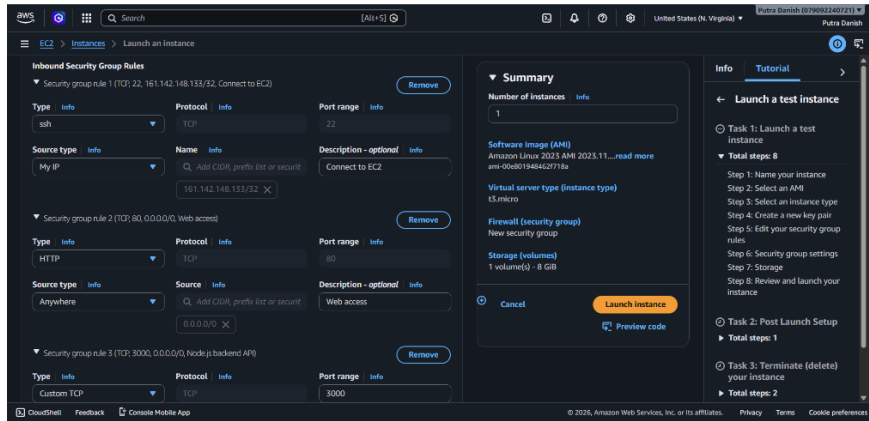


Figure 13: AWS Security Group Configuration

AWS Security Groups were configured to allow secure access to the EC2 instance.

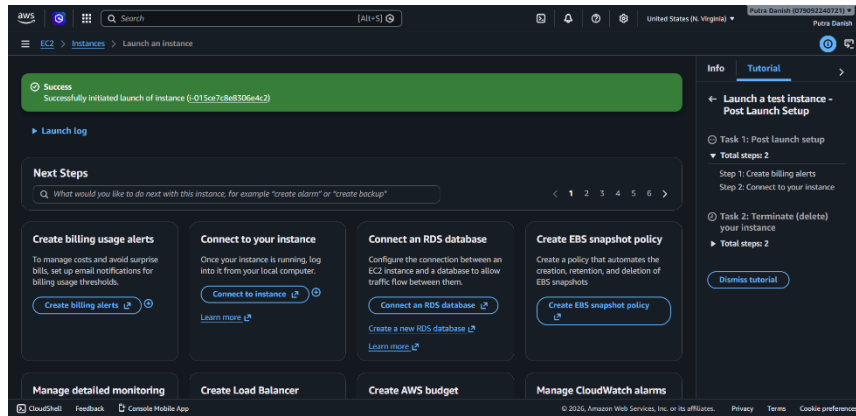


Figure 14: Amazon EC2 Instance Launch

The Amazon EC2 instance was successfully launched.

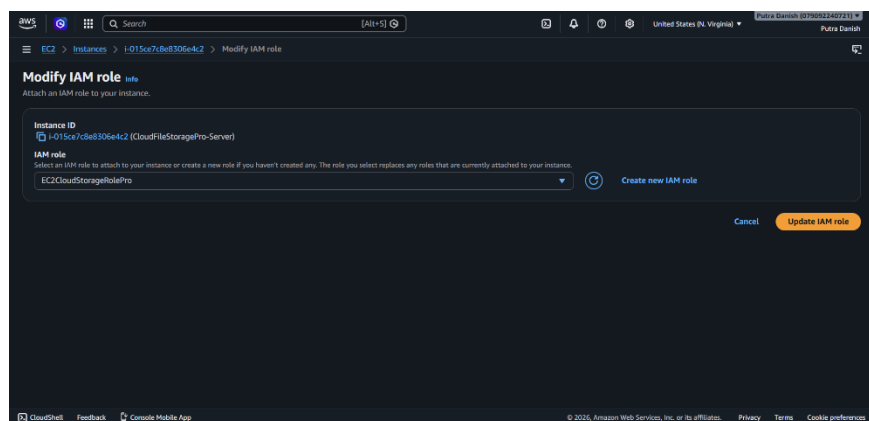


Figure 15: Modify IAM Role Assignment to EC2 Instance

An IAM role was assigned to the EC2 instance for secure access to AWS resources.

6.3 Connect to Amazon EC2 Instance

The Amazon EC2 instance was successfully connected using SSH and the downloaded key pair (.pem file). This connection allows administrators to access and manage the server remotely through the terminal.

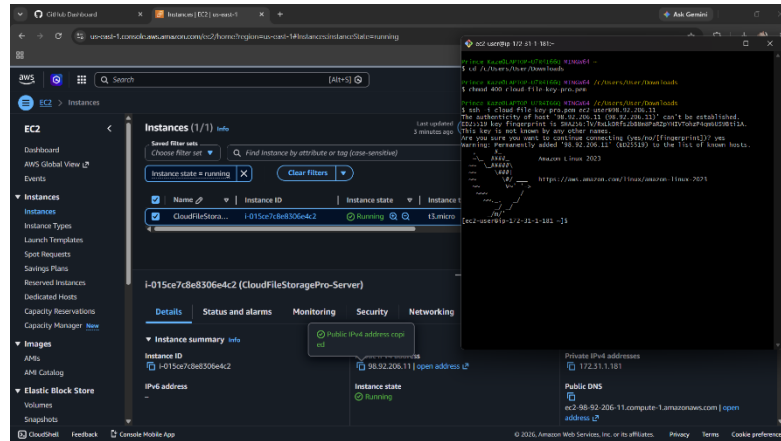


Figure 16: Connecting to Amazon EC2 Instance

The EC2 instance was successfully connected through SSH for remote server management using public IP Address.

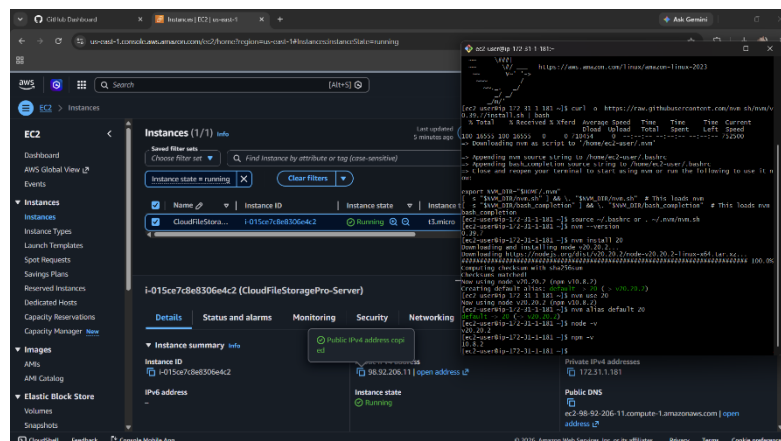


Figure 17: NVM and Node.js is installed to EC2 with verify each version

NVM is installed to manage Node.js versions easily on the EC2 instance. Node.js is installed through NVM and set as the default version. npm (Node Package Manager) installs required project dependencies such as Express, AWS SDK, Multer, and CORS. The EC2 instance is successfully configured as a Node.js server and is ready to host the AWS Cloud Storage application.

6.4 Backend Application Development and Configuration

This step involves creating the backend application environment on the Amazon EC2 instance. A Node.js project was initialized, required packages were installed, the server application was developed, and the backend service was successfully deployed and tested through a web browser.

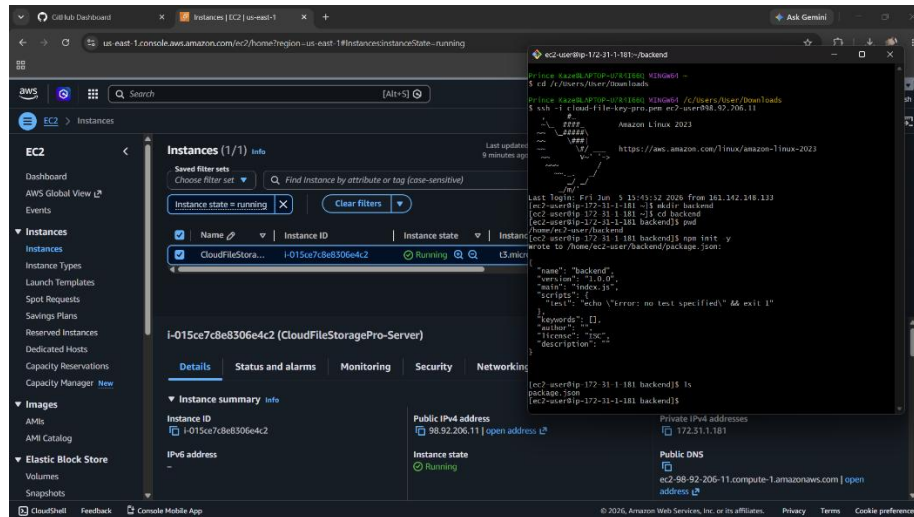


Figure 18: Backend Project Initialization

A new directory is created to store all backend application files and the package.json file was generated using npm.

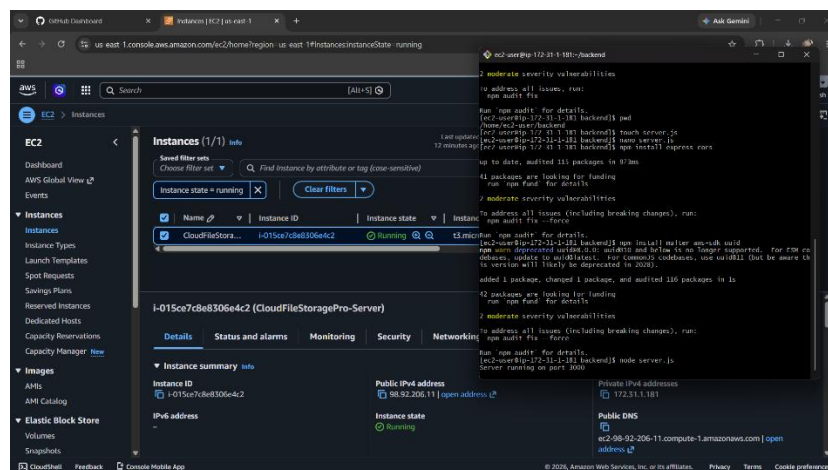


Figure 19: Installation of Backend Dependencies

Required Node.js packages were installed to support backend application development, which is Express, AWS SDK, Multer, CORS and UUID.

```
Run 'npm audit' for details.  
[ec2-user@ip-172-31-1-181 backend]$ node server.js  
Server running on port 3000
```

Figure 20: Backend Server Execution

The backend server was started successfully on port 3000 using the Node.js runtime.

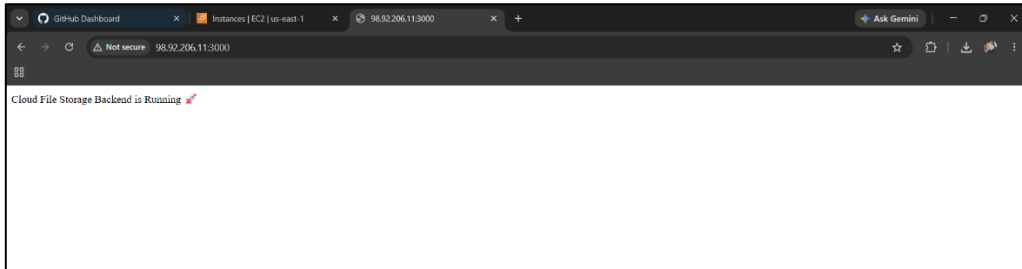


Figure 21: Backend Application Verification

The backend application was accessed through a web browser to verify that the server was running correctly.

6.5 Backend Integration and Application Testing

This step involves developing backend source code, integrating AWS services, configuring application modules, and testing the backend application to ensure successful deployment on the EC2 instance.

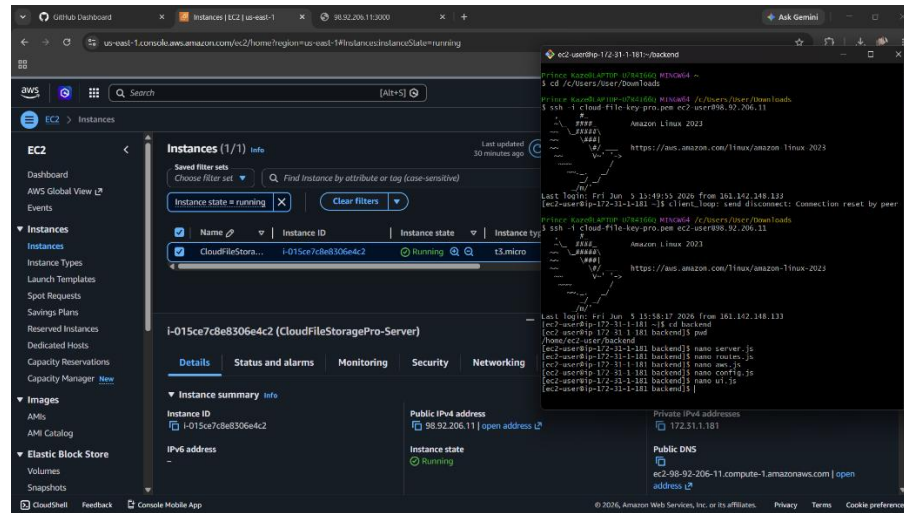


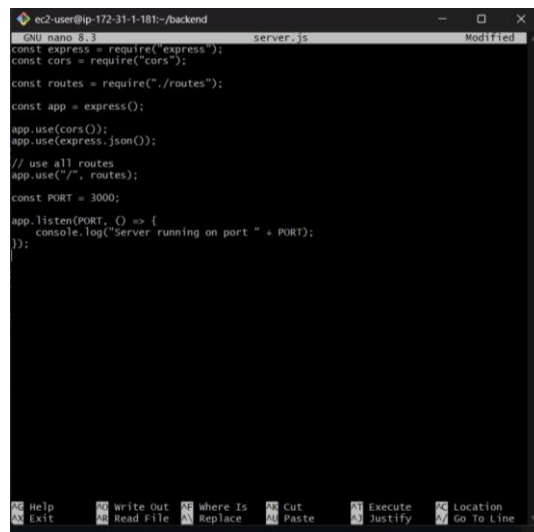
Figure 22: Backend Server Development

File Architecture:

```
backend/  
server.js  
routes.js  
aws.js  
config.js  
ui.js
```

a. server.js

The server.js file was created and configured to handle backend requests and start the application server.



```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3 server.js Modified
const express = require("express");
const cors = require("cors");

const routes = require("./routes");

const app = express();

app.use(cors());
app.use(express.json());

// use all routes
app.use("/", routes);

const PORT = 3000;

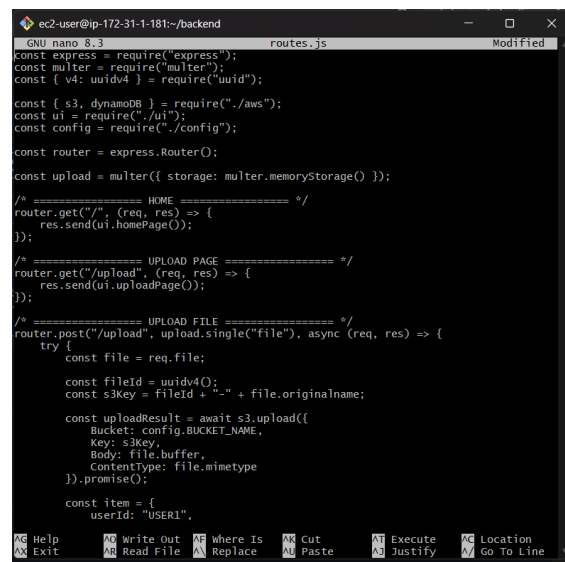
app.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line
```

Figure 23: server.js Configuration

b. routes.js

Handles all application routes and business logic. Import Express router, multer, uuid. Module export to Connect to server.js



```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3 routes.js Modified
const express = require("express");
const multer = require("multer");
const { v4: uuidv4 } = require("uuid");

const { s3, dynamoDB } = require("./aws");
const ui = require("./ui");
const config = require("./config");

const router = express.Router();

const upload = multer({ storage: multer.memoryStorage() });

/* ===== HOME ===== */
router.get("/", (req, res) => {
  res.send(ui.homePage());
});

/* ===== UPLOAD PAGE ===== */
router.get("/upload", (req, res) => {
  res.send(ui.uploadPage());
});

/* ===== UPLOAD FILE ===== */
router.post("/upload", upload.single("file"), async (req, res) => {
  try {
    const file = req.file;

    const fileId = uuidv4();
    const s3Key = fileId + "-" + file.originalname;

    const uploadResult = await s3.upload({
      Bucket: config.BUCKET_NAME,
      Key: s3Key,
      Body: file.buffer,
      ContentType: file.mimetype
    }).promise();

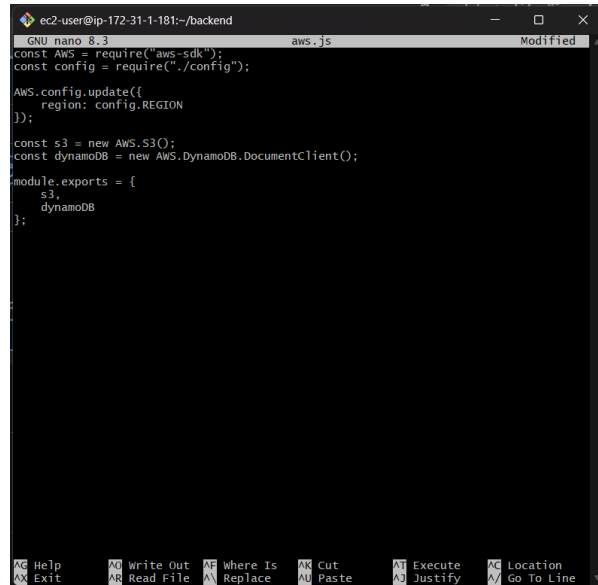
    const item = {
      userId: "USER1",

```

Figure 24: routes.js Configuration

c. aws.js

The aws.js file was configured to establish connections between the application and AWS services such as Amazon S3 and DynamoDB. Import aws-sdk and config create. S3 Bucket and DynamoDB connection. Export S3 and DynamoDB to AWS Services.



```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3      aws.js      Modified
const AWS = require("aws-sdk");
const config = require("../config");

AWS.config.update({
  region: config.REGION
});

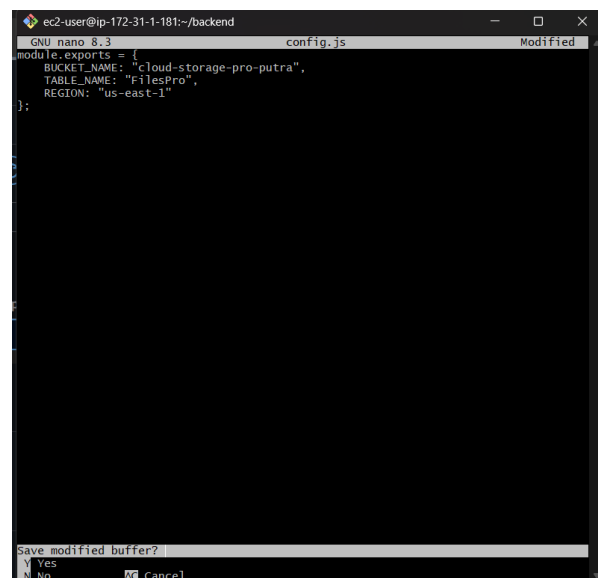
const s3 = new AWS.S3();
const dynamoDB = new AWS.DynamoDB.DocumentClient();

module.exports = {
  s3,
  dynamoDB
};
```

Figure 25: aws.js Configuration

d. config.js

Stores AWS configuration values used throughout the application. Centralizes configuration settings, making the application easier to maintain and update. Stored Information: S3 Bucket Name, DynamoDB Table Name and AWS Region.



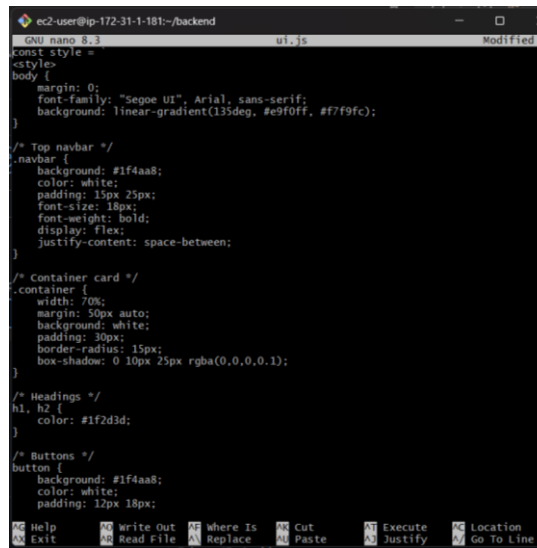
```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3      config.js    Modified
module.exports = {
  BUCKET_NAME: "cloud-storage-pro-putra",
  TABLE_NAME: "FilesPro",
  REGION: "us-east-1"
};

Save modified buffer?
Yes
No      Cancel
```

Figure 26: config.js Configuration

e. ui.js

Generates the web interface using HTML templates. Generate Interface: Home Page, Upload Page, Files View Page and Upload Success Page.



```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3 ui.js
const style =
<style>
body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: linear-gradient(135deg, #e9f0ff, #f7f9fc);
}

/* Top navbar */
.navbar {
  background: #1f4aa8;
  color: white;
  padding: 15px 25px;
  font-size: 18px;
  font-weight: bold;
  display: flex;
  justify-content: space-between;
}

/* Container card */
.container {
  width: 70%;
  margin: 50px auto;
  background: white;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.1);
}

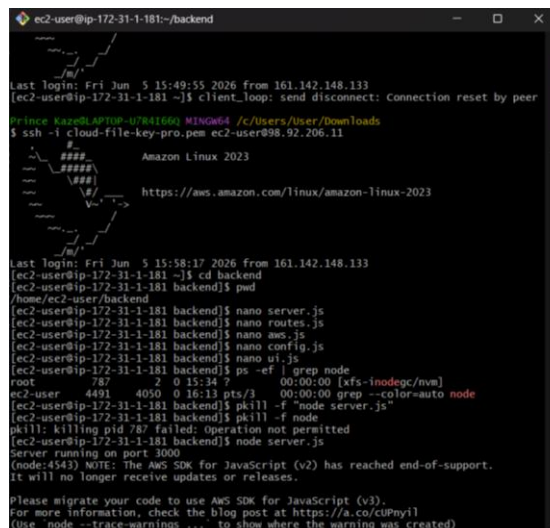
/* Headings */
h1, h2 {
  color: #1f2d3d;
}

/* Buttons */
button {
  background: #1f4aa8;
  color: white;
  padding: 12px 18px;
```

Figure 27: ui.js Configuration

Server Management Commands:

Check Running Node.js Process. Shows whether the Node.js server is running or not. Stops the currently running version of the application. Starts the Express application.



```
ec2-user@ip-172-31-1-181:~/backend
Last login: Fri Jun 5 15:49:55 2026 from 161.142.148.133
[ec2-user@ip-172-31-1-181 ~]$ client_loop: send disconnect: Connection reset by peer
Prince Kaze@LAPTOP-U7RA1660 MINGW64 /C:/Users/User/Downloads
$ ssh -i cloud-file-key-pro.pem ec2-user@98.92.206.11
#####
Amazon Linux 2023
#####
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Fri Jun 5 15:58:17 2026 from 161.142.148.133
[ec2-user@ip-172-31-1-181 ~]$ cd backend
[ec2-user@ip-172-31-1-181 backend]$ pwd
/home/ec2-user/backend
[ec2-user@ip-172-31-1-181 backend]$ nano server.js
[ec2-user@ip-172-31-1-181 backend]$ nano routes.js
[ec2-user@ip-172-31-1-181 backend]$ nano aws.js
[ec2-user@ip-172-31-1-181 backend]$ nano config.js
[ec2-user@ip-172-31-1-181 backend]$ nano ui.js
[ec2-user@ip-172-31-1-181 backend]$ ps -ef | grep node
root      787      2  0 15:34 ?        00:00:00 [xfs-inodegc/nw]
ec2-user  4491    4090  0 16:13 pts/3    00:00:00 grep --color=auto node
[ec2-user@ip-172-31-1-181 backend]$ kill -f "node server.js"
[ec2-user@ip-172-31-1-181 backend]$ kill -f node
kill: killing pid 787 failed: operation not permitted
[ec2-user@ip-172-31-1-181 backend]$ node server.js
Server running on port 3000
(node:4543) NOTE: The AWS SDK for JavaScript (v2) has reached end-of-support.
It will no longer receive updates or releases.
Please migrate your code to use AWS SDK for JavaScript (v3).
For more information, check the blog post at https://a.co/cUPmy11
(Use node --trace-warnings ... to show where the warning was created)
```

Figure 28: Server management and node server.js execute

The backend application is successfully developed using Node.js and Express.js. It provides complete functionality for uploading, viewing, and deleting files while integrating with Amazon S3 for cloud storage and DynamoDB for metadata management. Also, executed successfully on the EC2 instance and started listening on port 3000.

6 FINAL SYSTEM TESTING

The EC2 public IP address was successfully used to access the AWS Cloud Storage System. The web application displays the main interface for file management, including file upload functionality, uploaded file listings, file access links, and file deletion options. This verifies that the Node.js server, Express.js application, Amazon S3 storage service, Amazon DynamoDB database, and AWS cloud infrastructure were all operating correctly after deployment on Amazon EC2.

a. Home Page Web

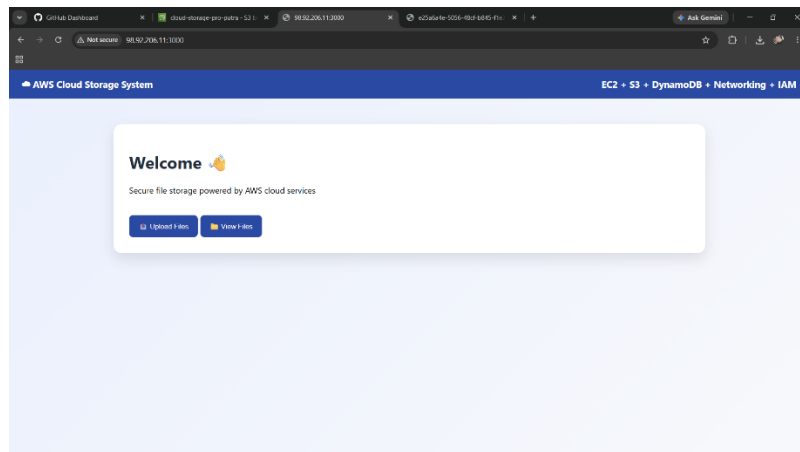


Figure 29: Home Page Testing

The home page was accessed through the EC2 public IP address and port 3000.

b. Upload File Operation

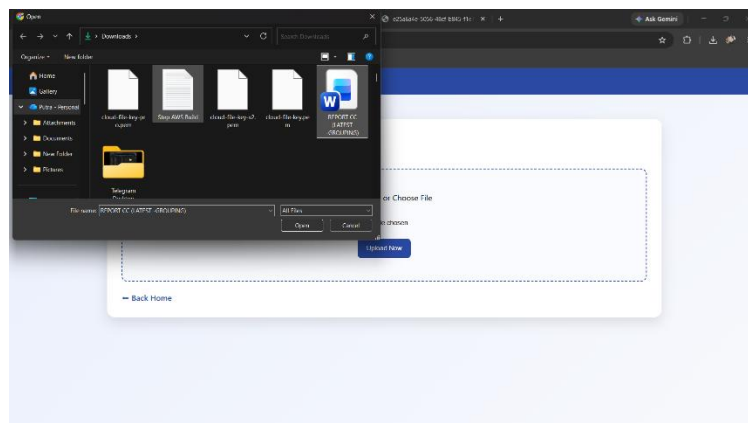


Figure 30: Upload Page Testing

The upload page was opened to test the file upload function.

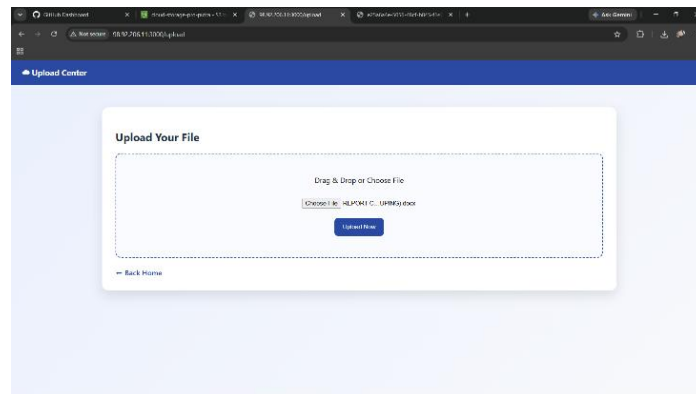


Figure 31: File Selection

A file was selected from the local computer before uploading.

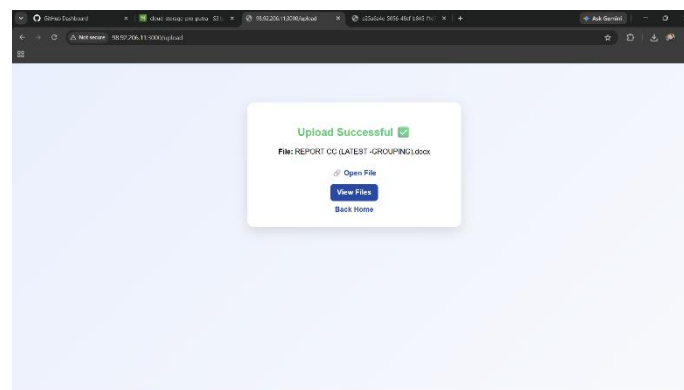


Figure 32: Upload Process

The selected file was uploaded through the web application. The system displayed an upload success message after the file was uploaded.

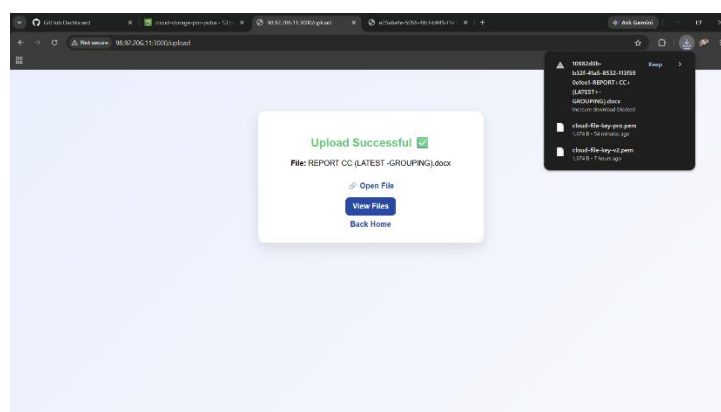


Figure 33: Upload Successful Page

Some file will be automatically downloaded to computer when click open file.

Amazon Verification for S3 Bucket and DynamoDB database:

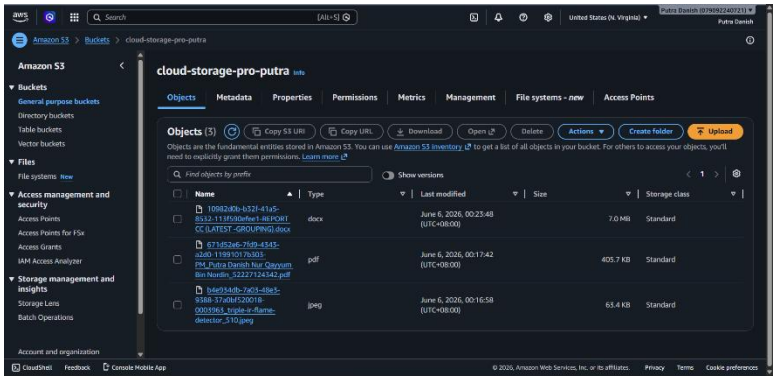


Figure 34: Amazon S3 File Storage Verification

The uploaded files are successfully stored in the Amazon S3 bucket, confirming that the upload process works correctly.

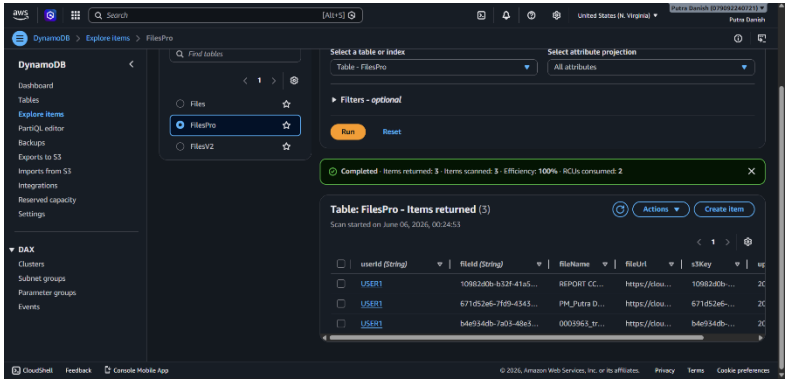


Figure 35: DynamoDB Metadata Verification

The file metadata is successfully stored in DynamoDB, confirming that the system records file information correctly after upload.

c. View Files

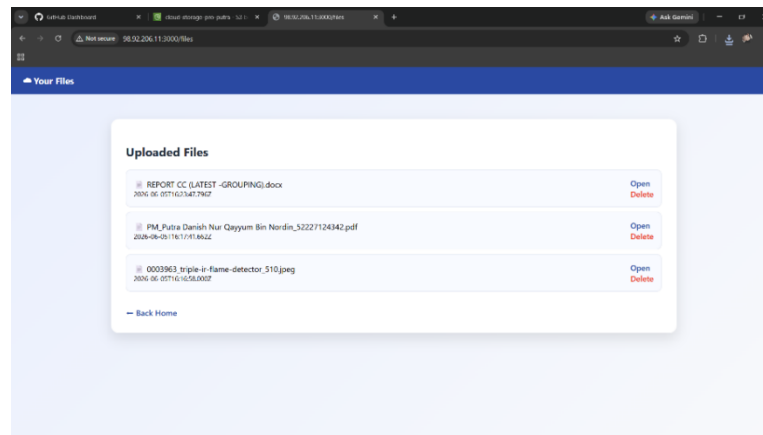


Figure 36: View Files List

The uploaded files were displayed successfully on the website. User can see their uploaded files on the View List Page.

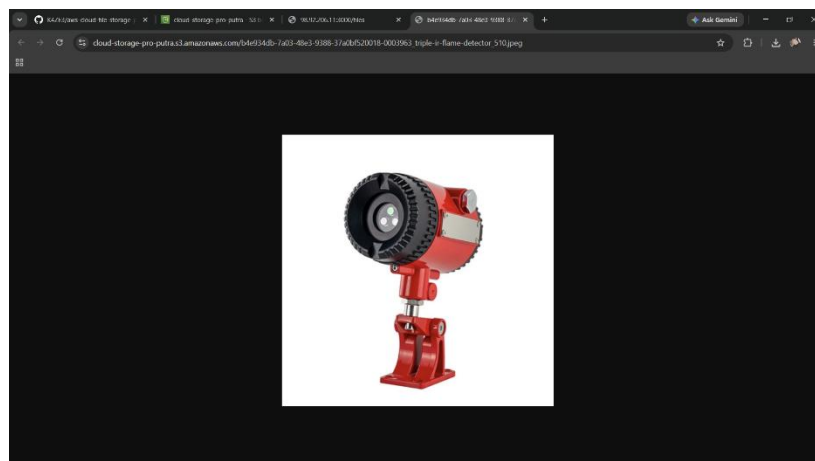


Figure 37: Open File from View List Page

Some file will be opened through the browser when click open file. So, user can see and access it immediately.

d. Delete File Operation

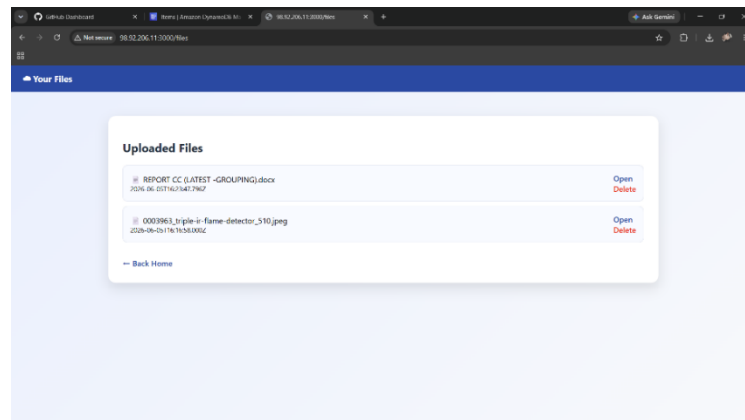


Figure 38: Deleted Files on Website

The deleted files are successfully removed from list view.

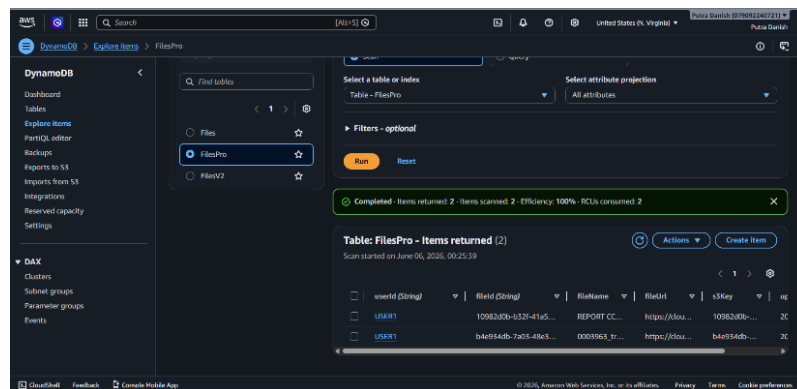


Figure 39: DynamoDB Deletion Verification

The number of records in DynamoDB has decreased, confirming that the selected file metadata was successfully removed after the delete operation.

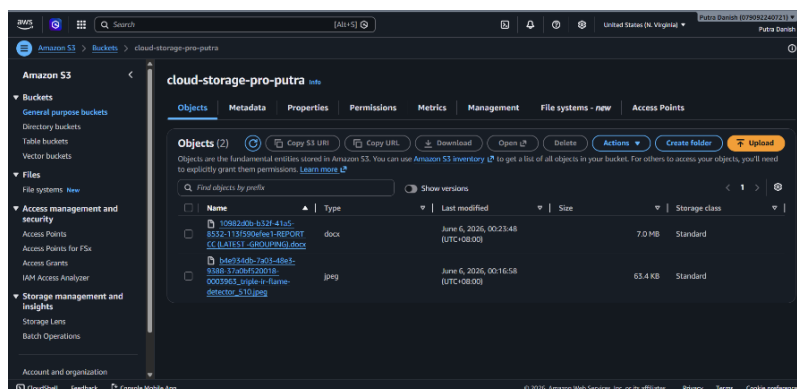


Figure 40: Amazon S3 Deletion Verification

The deleted file is no longer displayed in the Amazon S3 bucket, confirming that the file was successfully removed from cloud storage.

e. GitBash Terminal Verification:

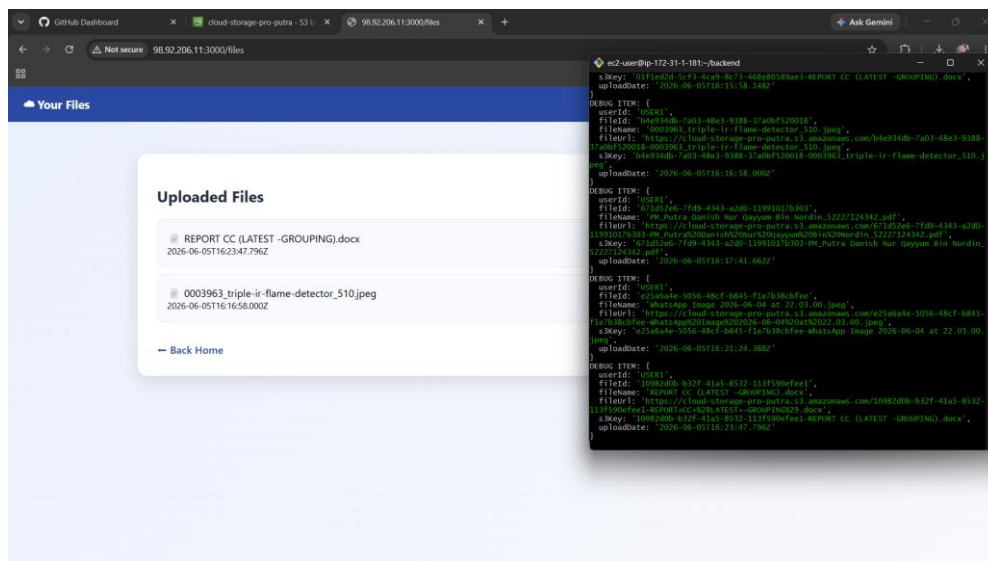


Figure 41: Git Bash terminal Verification

The GitBash successfully retrieves file information from S3 Bucket and DynamoDB and displays the metadata for each uploaded file.

7. GitHub Integration and Version Control

This phase demonstrates the integration of the AWS Cloud Storage project with GitHub. The project source code was committed and uploaded to a GitHub repository, providing version control, project backup, and easier collaboration.

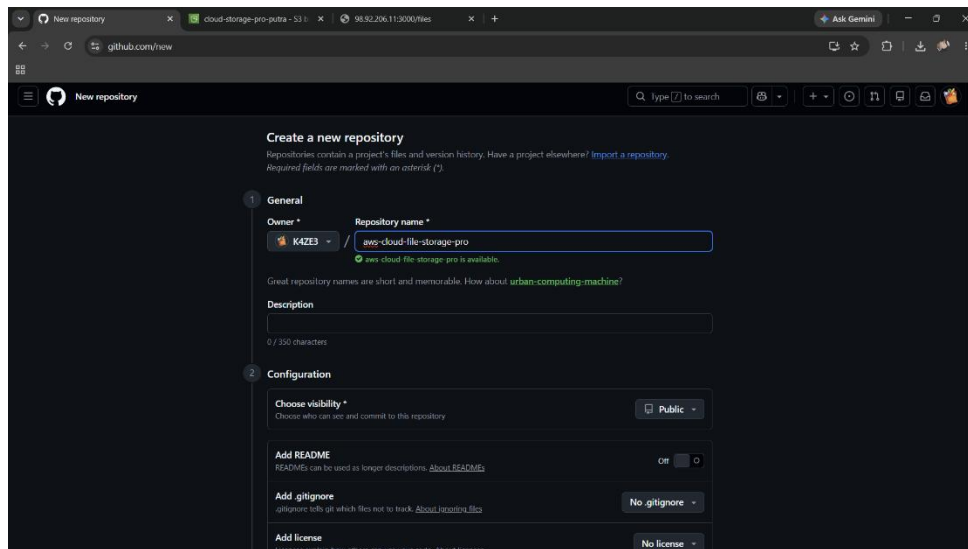


Figure 42: Generate GitHub Personal Access Token

A GitHub Personal Access Token (PAT) was created to authenticate the EC2 instance with GitHub.

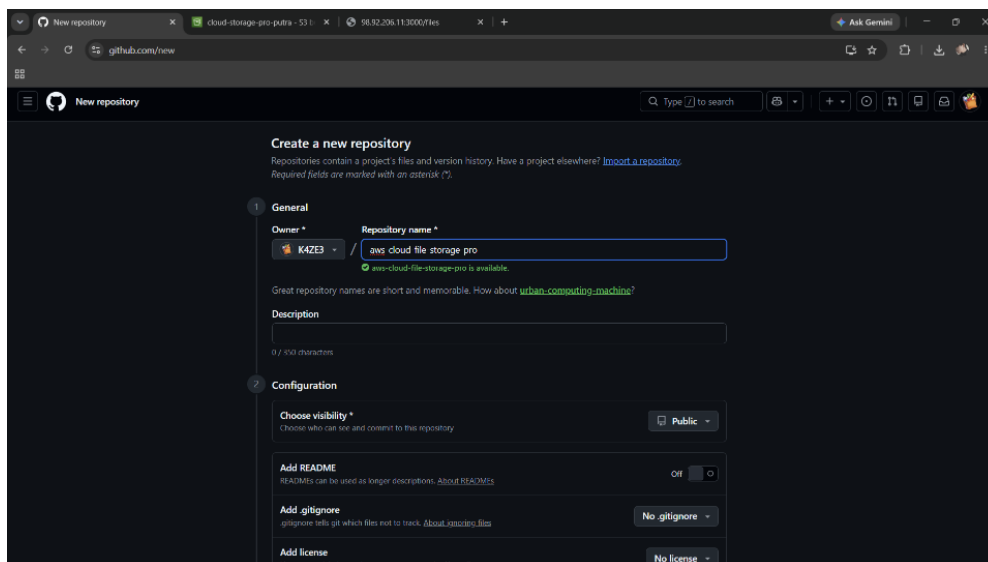


Figure 43: Create GitHub Repository

A new GitHub repository named **aws-cloud-file-storage-pro** was created to store the project source codes and files.

```
ec2-user@ip-172-31-1-181:~/backend
Installed:
git-2.50.1-1.amzn2023.0.1.x86_64
git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-error-1:0.17030-2.amzn2023.0.1.noarch
perl-file-find-1.37-477.amzn2023.0.8.noarch
perl-git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64
perl-lib-0.65-477.amzn2023.0.8.x86_64

Complete!
[ec2-user@ip-172-31-1-181 backend]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-1-181 backend]$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/ec2-user/backend/.git/
[ec2-user@ip-172-31-1-181 backend]$ git add
[ec2-user@ip-172-31-1-181 backend]$ git commit -m "AWS Cloud File Storage project"
[master (root-commit) 7566fe9] AWS Cloud File Storage project
Committer: EC2 Default User <ec2-user@ip-172-31-1-181.ec2.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:
    git config --global --edit
After doing this, you may fix the identity used for this commit with:
    git commit --amend --reset-author
3512 files changed, 2400974 insertions(+)
```

Figure 44: Initialize Local Git Repository

Git was initialized on the EC2 instance and the project files were committed.

```
ec2-user@ip-172-31-1-181:~/backend
GNU nano 8.3 .gitignore Modified
node_modules/
*.pem
*.env
```

Figure 45: Add .gitignore file

The .gitignore file was added to the repository of the GitHub.

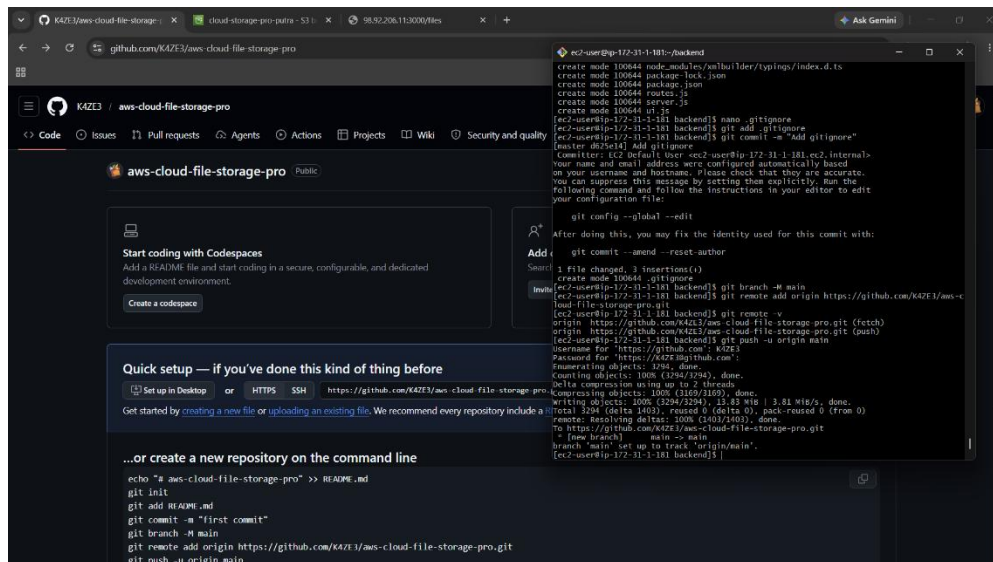


Figure 46: Connect and Push Repository to GitHub

The local repository was connected to GitHub and all project files and codes were pushed successfully.

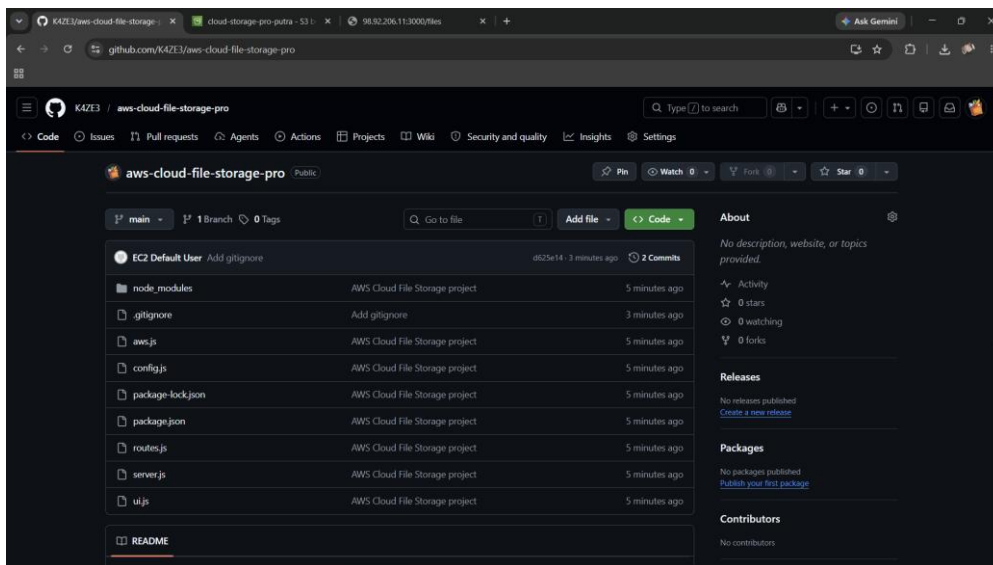


Figure 47: GitHub Repository Successfully Uploaded

The project files were successfully uploaded and displayed in the GitHub repository.

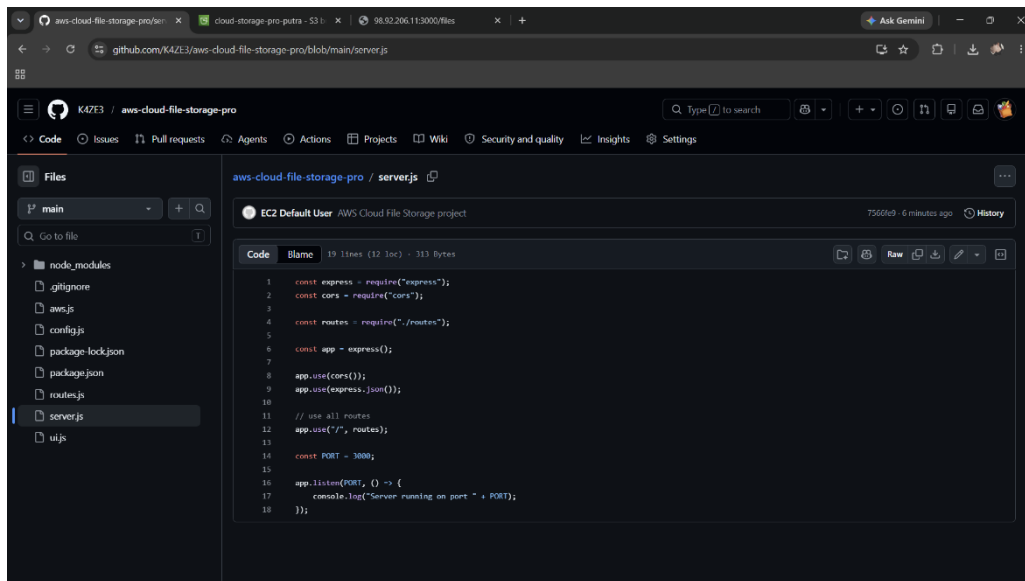


Figure 48: Source Code Verification in GitHub

The uploaded source code files can be accessed and viewed directly from GitHub.

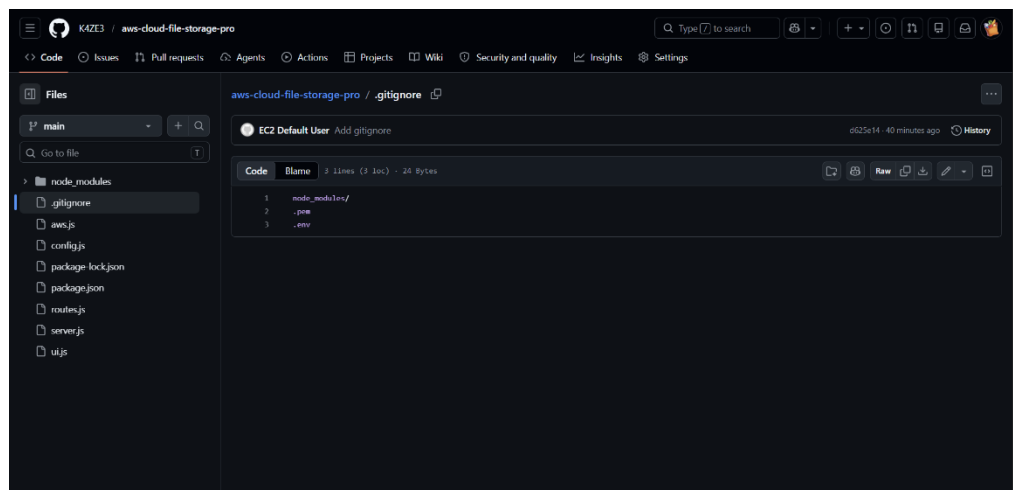


Figure 49: .gitignore Verification

The .gitignore file was successfully uploaded to GitHub to exclude sensitive and unnecessary files.

8 RESULTS

The AWS Cloud Storage System was successfully developed, tested, and deployed on the AWS cloud platform using an Amazon EC2 instance running Amazon Linux. After deployment, the system was accessible through the EC2 public IPv4 address using a web browser. The web application functioned as expected and all major features were tested successfully.

Users were able to upload files to Amazon S3 cloud storage through the web application. Uploaded files were successfully stored in the S3 bucket, while corresponding file metadata including file names, file URLs, upload dates, file identifiers, and storage keys were recorded in Amazon DynamoDB. Users were also able to view uploaded files, access files through generated URLs, and delete files from both Amazon S3 and DynamoDB when required.

The Node.js server and Express.js web application were successfully configured and deployed on the EC2 instance, allowing the system to process user requests and communicate efficiently with AWS cloud services. AWS Security Groups were configured to provide secure access to the application through the required network ports, while AWS Identity and Access Management (IAM) was implemented to manage permissions and secure access to AWS resources.

Amazon S3 and Amazon DynamoDB were successfully integrated with the web application, demonstrating the practical use of AWS storage and database services within a cloud computing environment. The successful interaction between EC2, S3, DynamoDB, IAM, and Security Groups confirmed the effectiveness of the cloud architecture implemented for the project.

Overall, the project successfully demonstrated the practical implementation of cloud computing concepts by integrating AWS Compute (Amazon EC2), Storage (Amazon S3), Database (Amazon DynamoDB), Networking (Public IP Address and Security Groups), and Security (IAM) services into a functional Cloud Storage System. The project achieved all objectives and provided valuable hands-on experience in AWS deployment, cloud infrastructure management, web application hosting, and cloud service integration.

7 CONCLUSION

In conclusion, the AWS Cloud Storage System was successfully developed and deployed using AWS cloud services. Amazon EC2 was utilized to host the Node.js and Express.js web application, Amazon S3 was used to store uploaded files, while Amazon DynamoDB was implemented to store and manage file metadata. AWS IAM and Security Groups were configured to ensure secure access and management of cloud resources.

The system successfully achieved its objectives by providing functionalities such as file uploading, viewing, accessing, and deletion through a web-based interface. The application was tested both locally and in the AWS cloud environment, and all core features operated as expected. The deployment process also demonstrated the successful integration of multiple AWS services into a fully functional cloud-based file storage solution accessible through the internet.

Furthermore, the project provided valuable hands-on experience in cloud computing technologies, including server configuration, web application hosting, cloud storage management, database integration, networking, and security management. Through the integration of AWS Compute, Storage, Networking, Database, and Security services, the project demonstrated how cloud platforms can be used to build scalable, secure, and reliable web applications.

Overall, the AWS Cloud Storage System serves as a practical example of applying cloud computing concepts in a real-world scenario and has enhanced the team's understanding of AWS services, cloud infrastructure, and cloud-based application deployment.

REFERENCES

1. Amazon Web Services. (2025). *Amazon EC2 documentation*. AWS. <https://docs.aws.amazon.com/ec2/>
2. Amazon Web Services. (2025). *Amazon Simple Storage Service (Amazon S3) documentation*. AWS. <https://docs.aws.amazon.com/s3/>
3. Amazon Web Services. (2025). *Amazon DynamoDB documentation*. AWS. <https://docs.aws.amazon.com/dynamodb/>
4. Git. (2025). *Git documentation*. <https://git-scm.com/doc>
5. Mozilla Developer Network. (2025). *Express tutorial: Express/Node introduction*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction
6. Express.js. (2025). *Express web application framework documentation*. <https://expressjs.com/>

YouTube Link: <https://youtu.be/kZ4MyGgalvk?si=pRumJMPhasV4L9Kn>

GitHub Repository Link: <https://github.com/K4ZE3/aws-cloud-file-storage-pro>